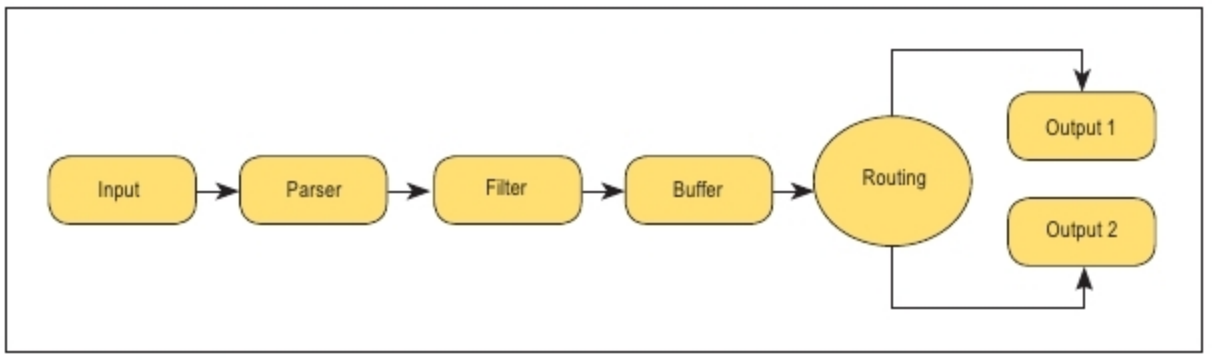

Figure 4: Fluent Bit data pipeline

The parser allows us to convert the incoming data to structured data so that data operations can be faster. For example, the JSON parser plugin takes the JSON map string and converts it directly into the internal binary representation.

Filtering is implemented through various plugins, and they can be used to modify our logs, append additional information to our original logs or drop certain logs. It basically gives us a chance to alter our data before we send it to a destination. There are many filter plugins. Some of them, like the Lua Filter, allow us to modify the incoming records using custom Lua scripts.

The buffer phase comes after filtering. In general, a data buffer is a region of a physical memory storage used to temporarily store data while it is being moved from one place to another. The Fluent Bit buffer is exactly that. It provides a unified and persistent way to store data. The data in the buffer cannot be changed, i.e., it is immutable.

After the buffer, Fluent Bit routes the data through filters to various destinations. The router relies on the concept of tags and matching rules. Tag and matching have been explained earlier in key concepts.

The final stage in the data pipeline is the output stage, at which point Fluent Bit sends the record to the destination. Our destination can be databases, cloud services or even standard output. There are many output plugins like Azure, ElasticSearch, InfluxDB, etc.

Now let us discuss the setup configuration of our own end-to-end data pipeline.

Setting up your own pipeline

Let us suppose you want to send JSON records to the standard output but you also want every record to have new key/value pairs. The record should contain the original keys plus a new record called *tag*.

Make a new configuration file in the */conf directory,* which is the source repository that you installed earlier and name it as *my_configuration.conf*. Note that this works at the global scope.

Any configuration file first needs a *SERVICE* section that defines the global properties of the service. Here is an example of the section:

```
1. [SERVICE]
2. # Set an interval of seconds before
flushing records to a destination
3. Flush        5
4. # Tells Fluent Bit whether to run
as a daemon or not
5. Daemon       Off
6. # Set the verbosity level of the
service.
7. Log_Level    info
```

Next, we define the *INPUT* section, which is a dummy JSON record here that shall be further used in the pipeline:

```
8.  [INPUT]
9.  # Name of plugin
10.  Name dummy
11.  # Dummy JSON string
12.  Dummy {"this is":"dummy data"}
```

Now we need to append a new record called *tag* to the original record as well as a time stamp. Basically, in this step, we are modifying the record. This can be done by filtering the record. So, we define a *FILTER* section. We filter the record using a custom Lua script in the example given below. It allows for a flexible filtering mechanism. A Lua based filter has two steps. First, we need to configure the filter in the *FILTER* section in the main configuration file (in this case *my_configuration.conf*) and then prepare a Lua script to be used by the filter.

```
13.  [FILTER]
14.  # Name of filter
15.  Name lua
16.  Match *
17.  # Path to the script
18.  script /home/atibhi/fluent-bit/
scripts/append_tag.lua
19.  # Lua function name defined inside
script that will be triggered.
20.  call append_tag
```

The Lua script is as follows:

```
function append_tag (tag, timestamp,
record)
   new_record = record
   new_record["tag"] = tag
   return 1, timestamp, new_record
end
```

The function contains three arguments. 'Tag' is associated with incoming input, 'timestamp' is the input and 'record' has the record contents. It then appends the *tag* to the incoming record and returns the modified record. Finally, after filtering, we want our record to go to the output destination, which in this case is the standard output. Next, the *OUTPUT* section has to be defined.

```
21.  [OUTPUT]
22.  # Name of output plugin
```